AEM Educational Institute

Case Study Documentation

Complete Technical Reference

Adobe Experience Manager 6.5
Cognizant GenC AEM Training Program

Project	GenC AEM Training - Educational Institute Site
AEM Version	6.5
Java Version	Java 11
Build Tool	Apache Maven
Document Version	1.0
Last Updated	May 14, 2026

Table of Contents

1. Project Overview

This document captures the complete technical implementation of the **Educational Institute Site** built as part of the Cognizant GenC AEM Training case study. The objective was to build a fully functional AEM 6.5 website demonstrating core AEM concepts: content modeling, custom components, OSGi services, Sling Models, Experience Fragments, editable templates, and role-based access control.

Project Goals

Build an educational institute website with the following capabilities:

- **Course listing** driven by Content Fragments and a configurable OSGi service
- **Custom component** (Course Cards) that displays courses with filter, sort, and limit options
- **Site-wide header and footer** managed through Experience Fragments
- **Custom Header/Footer components** with multifield dialogs for navigation links
- **Four pages**: Home, About Us, Courses, Contact Us — all using a common template
- **Role-based access control** with three groups (authors, XF admins, readers)

High-Level Architecture

Layer	Implementation
Content	Tags + Content Fragment Model + 12 Content Fragments
Configuration	OSGi Annotation + Service Interface + Implementation + .cfg.json
Business Logic	Sling Model (CourseListModel) + POJO (CourseBean)
Presentation	Custom Components (Course Cards, Header, Footer) with HTL + Dialogs
Styling	Clientlibs (CSS/JS bundles)
Composition	Experience Fragments + Editable Template
Security	User/Group/ACL based RBAC

Project Structure (Maven Modules)

```
genc-aem-training/
■■■ core/                    # Java code (Sling Models, OSGi services)
■■■ ui.apps/                 # Components, clientlibs, templates
■■■ ui.apps.structure/       # Allowed roots for ui.apps
■■■ ui.config/               # OSGi configurations (.cfg.json)
■■■ ui.content/              # CF Model, pages, tags
■■■ all/                     # Aggregates all packages
■■■ it.tests/                # Integration tests
■■■ ui.tests/                # Cypress UI tests (skipped due to SSL)
```

2. Tags & Taxonomies

Concept

AEM Tags allow content to be categorized using a hierarchical taxonomy. Tags are stored as `cq:Tag` nodes under `/content/cq:tags/`. Each tag has a unique tag ID composed of namespace and path (e.g., `genc-aem_training:course-category/engineering`).

What Was Built

Element	Value
Namespace title	Genc AEM Training
Namespace path	/content/cq:tags/genc-aem_training
Parent tag	course-category
Number of child tags	10
Categories	Engineering, Management, Arts, Science, Commerce, Medical, Law, IT, Design, Educ

Note on Namespace Naming

■ **Note:** AEM auto-converts hyphens to underscores in tag namespace names. The intended name was **genc-aem-training** (hyphens), but the actual node name became **genc-aem_training** (underscore). All Content Fragment Model references, Sling Model code, and dialog configurations use the underscore variant.

Tag ID Format

```
# Format: <namespace>:<parent>/<child>
genc-aem_training:course-category/engineering
genc-aem_training:course-category/management
genc-aem_training:course-category/it
# ... and 7 more
```

Verification URL

```
http://localhost:4502/aem/tags.html/content/cq:tags/genc-aem_training
```

Demo Talking Point

> *"Tags allow content categorization. I created a namespace under cq:tags to scope the Course Category vocabulary to this project, with 10 child tags representing course categories. The tag picker in the Course Cards dialog is constrained to this namespace, ensuring authors can only select from the predefined categories."*

3. Content Fragment Model (CF Model)

Concept

A Content Fragment Model is a reusable schema that defines the structure of content-as-data — separating content from presentation. Each model is a Java-class-like blueprint with typed fields. Multiple fragments can be created from one model.

What Was Built

Property	Value
Model name	Course
Path	/conf/genc-aem-training/settings/dam/cfm/models/course
Number of fields	6

Fields Defined

Field	Type	Required	Notes
courseTitle	Single-line Text	Yes	Display name
courseDescription	Multi-line Text (Rich Text)	Yes	Stored as HTML
startDate	Date and Time	Yes	Stored as Calendar
endDate	Date and Time	Yes	Stored as Calendar
courseCategory	Tags (single-select)	Yes	Rooted at course-category
courseImage	Content Reference	No	Optional image asset

Tags Field Configuration

The Course Category field's root path is set to `/content/cq:tags/genc-aem_training/course-category` so the tag picker only shows the 10 relevant tags instead of the entire AEM tag tree.

Demo Talking Point

> *"I built a Course content fragment model with 6 strongly-typed fields. CF Models separate content structure from presentation — the same Course data can be rendered by different components (cards, list, table) without changing the data. The Tags field is scoped to my custom taxonomy via a root path restriction."*

4. Content Fragments (CFs)

Concept

Content Fragments are instances of data based on a CF Model — like objects of a class. They live in the DAM (Digital Asset Manager) as nodes of type `dam:Asset`. They have a special property `cq:model` linking back to the model.

What Was Built

12 course content fragments, all under `/content/dam/genc-aem-training/courses/`:

#	Title	Node Name	Category
1	B.Tech Computer Science	b-tech-computer-science	Engineering
2	MBA Finance	mba	Management
3	BSc Physics	bsc-physics	Science
4	B.Com	b-com	Commerce
5	MBBS	mbbs	Medical
6	LLB	llb	Law
7	B.Des UI/UX	b-des-ui-ux	Design
8	M.Tech AI/ML	mtech(ai&ml)	Engineering
9	BA English Literature	ba-english-literature	Arts
10	BCA	bca	IT
11	MSc Data Science	msc-data-science	IT
12	BBA	bba	Management

Key Property

Each fragment carries a `cq:model` property linking it back to the Course model. The Sling Model uses this property to identify Course fragments while traversing the DAM tree.

```
cq:model = /conf/genc-aem-training/settings/dam/cfm/models/course
```

Demo Talking Point

> *"Each course is a content fragment — pure data, decoupled from any visual presentation. Authors can update course content (titles, descriptions, dates) without touching code or layout. The fragment's cq:model property is what allows my Sling Model to identify and filter Course fragments from any other DAM content."*

5. OSGi Configuration & Service

Concept

OSGi is the runtime framework AEM uses to manage Java bundles. An OSGi **service** is a Java object registered in the OSGi service registry, available for dependency injection by other components. An OSGi **configuration** defines values that can be changed at runtime via the Web Console without recompiling code.

Architecture: 3 Files + 1 JSON Config

File	Purpose
CourseConfig.java	@interface defining configurable attributes
CourseConfigService.java	Interface exposing config values via getters
CourseConfigServiceImpl.java	Implementation registered as OSGi service
.cfg.json	Deployed config values per environment

CourseConfig.java (Annotation)

```
package com.genc.core.core.services;

import org.osgi.service.metatype.annotations.AttributeDefinition;
import org.osgi.service.metatype.annotations.ObjectClassDefinition;

@ObjectClassDefinition(
    name = "GenC AEM Training - Course Configuration",
    description = "Configuration for the Educational Institute course listing"
)
public @interface CourseConfig {

    @AttributeDefinition(
        name = "Course Root Path",
        description = "DAM root path where course CFs are stored"
    )
    String courseRootPath()
        default "/content/dam/genc-aem-training/courses";

    @AttributeDefinition(
        name = "Default Limit",
        description = "Default number of courses to display"
    )
    int defaultLimit() default 10;
}
```

CourseConfigService.java (Interface)

```
package com.genc.core.core.services;

public interface CourseConfigService {
    String getCourseRootPath();
    int getDefaultLimit();
}
```

CourseConfigServiceImpl.java (Implementation)

```
package com.genc.core.core.services.impl;

import org.osgi.service.component.annotations.Activate;
import org.osgi.service.component.annotations.Component;
import org.osgi.service.metatype.annotations.Designate;

import com.genc.core.core.services.CourseConfig;
import com.genc.core.core.services.CourseConfigService;

@Component(service = CourseConfigService.class, immediate = true)
@Designate(ocd = CourseConfig.class)
public class CourseConfigServiceImpl implements CourseConfigService {

    private String courseRootPath;
    private int defaultLimit;

    @Activate
    protected void activate(CourseConfig config) {
        this.courseRootPath = config.courseRootPath();
        this.defaultLimit = config.defaultLimit();
    }

    @Override
    public String getCourseRootPath() { return courseRootPath; }

    @Override
    public int getDefaultLimit() { return defaultLimit; }
}
```

.cfg.json (Deployed Configuration)

Path: `ui.config/.../osgiconfig/config/com.genc.core.core.services.impl.CourseConfigServiceImpl.cfg.json`

```
{
  "courseRootPath": "/content/dam/genc-aem-training/courses",
  "defaultLimit": 10
}
```

Key Annotations Explained

Annotation	Purpose
@ObjectClassDefinition	Marks the interface as an OSGi config schema. The name appears in the OSGi Web C
@AttributeDefinition	Each method becomes a configurable field with default value.
@Component	Registers the class as an OSGi service component.
@Designate	Binds the component to the metatype, enabling Web Console configuration.
@Activate	Method called when the component activates with resolved config.

Verification

Visit `http://localhost:4502/system/console/configMgr` and search for **"GenC AEM Training"**. The configuration appears with both fields pre-populated from the .cfg.json file.

Demo Talking Point

"The DAM root path is configurable via OSGi — administrators can change where the Sling Model looks for courses without redeploying code. Configuration is deployed as JSON files in source control for repeatability across environments. This separates deployment-time concerns from runtime concerns."

6. Sling Models

Concept

A Sling Model is a Java class that adapts a JCR Resource or HTTP Request into a clean Java object. It is the bridge between content (in JCR) and presentation (in HTL). Sling Models use dependency injection to access services, resources, and dialog values — making the code testable and decoupled.

Two Files Built

CourseBean.java (POJO)

A simple data container holding one course's data. Dates are pre-formatted as Strings in Java (rather than passing Calendar objects to HTL) because HTL has issues with Calendar truthy-checks.

```
package com.genc.core.core.models;

public class CourseBean {
    private String title;
    private String description;
    private String startDate;        // pre-formatted "dd MMM yyyy"
    private String endDate;
    private long startDateMillis;    // for sorting
    private String category;
    private String categoryTitle;
    private String path;

    // ... getters and setters for all fields
}
```

CourseListModel.java (the Sling Model)

Key annotations:

Annotation	Purpose
@Model(adaptables=...)	Declares this class as a Sling Model adaptable from Request and Resource
@ValueMapValue	Injects a property value from the current resource's ValueMap (dialog field)
@OSGiService	Injects an OSGi service by interface type
@Self	Injects the adaptable itself (the current request)
@PostConstruct	Marks a method called after all fields have been injected

What CourseListModel does in init():

- Reads the DAM root path from `CourseConfigService`
- Recursively walks the DAM folder structure
- For each asset, checks if `cq:model` matches Course model path
- Adapts to `ContentFragment` and reads each field
- Resolves tags via `TagManager.resolve()` with string-extraction fallback
- Filters by category if a filter is set in the dialog

- Sorts by start date ascending (using `startDateMillis`)
- Applies limit: dialog value takes precedence, falls back to OSGi default

Key Code Snippet - Model Declaration

```
@Model(
    adaptables = { SlingHttpServletRequest.class, Resource.class },
    defaultInjectionStrategy = DefaultInjectionStrategy.OPTIONAL
)
public class CourseListModel {

    @Self
    private SlingHttpServletRequest request;

    @ValueMapValue
    private Long limit;

    @ValueMapValue
    private String filterCategory;

    @ValueMapValue
    private String headingText;

    @OSGiService
    private CourseConfigService courseConfigService;

    @PostConstruct
    protected void init() {
        // ... read fragments, filter, sort, limit
    }
}
```

Key Code Snippet - Date Formatting in Java

Pre-formatting dates in Java avoids HTL's problematic truthy-checks on Calendar objects:

```
private String formatDate(Calendar cal) {
    SimpleDateFormat sdf = new SimpleDateFormat("dd MMM yyyy", Locale.ENGLISH);
    return sdf.format(cal.getTime());
}

// Usage in buildBean():
Calendar startCal = readCalendar(cf, "startDate");
if (startCal != null) {
    bean.setStartDate(formatDate(startCal));
    bean.setStartDateMillis(startCal.getTimeInMillis());
}
```

Key Code Snippet - Tag Resolution with Fallback

```
if (StringUtils.isNotBlank(category)) {
    String resolvedTitle = null;

    // Primary: use TagManager
    if (tagManager != null) {
        Tag tag = tagManager.resolve(category);
        if (tag != null) {
            resolvedTitle = tag.getTitle();
        }
    }
```

```
        // Fallback: extract last path segment and capitalize
        if (StringUtils.isBlank(resolvedTitle)) {
            int slashIdx = category.lastIndexOf('/');
            if (slashIdx > -1 && slashIdx < category.length() - 1) {
                String segment = category.substring(slashIdx + 1);
                resolvedTitle = capitalize(segment);
            }
        }
        bean.setCategoryTitle(resolvedTitle);
    }
```

Public API exposed to HTL

Getter	HTL Usage	Returns
getCourses()	${model.courses}	List<CourseBean>
getTotalCount()	${model.totalCount}	int
isEmpty()	${model.empty}	boolean
getHeadingText()	${model.headingText}	String

Demo Talking Point

> *"The Sling Model decouples content reading from rendering. It uses dependency injection to get the OSGi service and reads content fragments from DAM. Business logic (filtering, sorting, limiting) is applied in Java rather than HTL, keeping the template simple and the logic testable. Date formatting is also done in Java to avoid HTL limitations with Calendar objects."*

7. Course Cards Custom Component

Concept

An AEM component is a folder under `/apps/<app>/components/` containing files that together define how content renders. The folder name becomes the component name and is referenced via its **resource type**.

File Structure

```
coursecards/
■■■ .content.xml              # Component registration (cq:Component)
■■■ coursecards.html          # HTL template
■■■ _cq_dialog/
    ■■■ .content.xml          # Authoring dialog (Granite UI)
```

.content.xml (Component Definition)

```
<?xml version="1.0" encoding="UTF-8"?>
<jcr:root xmlns:cq="http://www.day.com/jcr/cq/1.0"
          xmlns:jcr="http://www.jcp.org/jcr/1.0"
    jcr:primaryType="cq:Component"
    jcr:title="Course Cards"
    jcr:description="Displays courses as cards with filter and limit."
    componentGroup="GenC AEM Training - Content"/>
```

coursecards.html (HTL Template)

```
<div class="cmp-coursecards"
     data-sly-use.model="com.genc.core.core.models.CourseListModel">

    <h2 class="cmp-coursecards__heading"
        data-sly-test="${model.headingText}">
        ${model.headingText}
    </h2>

    <div class="cmp-coursecards__empty"
         data-sly-test="${model.empty}">
        <p>No courses available at the moment.</p>
    </div>

    <div class="cmp-coursecards__count"
         data-sly-test="${!model.empty}">
        <p>Showing ${model.totalCount} courses</p>
    </div>

    <div class="cmp-coursecards__grid"
         data-sly-list.course="${model.courses}"
         data-sly-test="${!model.empty}">

        <article class="cmp-coursecards__card">
            <h3 class="cmp-coursecards__card-title">
                ${course.title}
            </h3>

            <div class="cmp-coursecards__card-category"
                 data-sly-test="${course.categoryTitle}">
                <span class="cmp-coursecards__badge">
                    ${course.categoryTitle}
```

```
                </span>
            </div>

            <div class="cmp-coursecards__card-dates">
                <span data-sly-test="${course.startDate}">
                    <strong>Start:</strong> ${course.startDate}
                </span>
                <span data-sly-test="${course.endDate}">
                     | <strong>End:</strong> ${course.endDate}
                </span>
            </div>

            <div class="cmp-coursecards__card-desc"
                 data-sly-test="${course.description}">
                ${course.description @ context='html'}
            </div>
        </article>
    </div>
</div>
```

Key HTL Constructs Used

Construct	Purpose
data-sly-use.model="..."	Instantiates the Sling Model
data-sly-test="${...}"	Conditionally renders the element
data-sly-list.x="${y}"	Iterates collection y as variable x
${expr @ context='html'}	Renders rich-text HTML safely

Dialog (_cq_dialog/.content.xml)

Three configurable fields:

Field	Type	Property
Heading Text	textfield	./headingText
Limit	numberfield (0-50)	./limit
Filter by Category	tagfield (rooted)	./filterCategory

Field-to-Sling-Model Mapping

Dialog Field	JCR Property	Sling Model Field
Heading Text	./headingText	@ValueMapValue String headingText
Limit	./limit	@ValueMapValue Long limit
Filter by Category	./filterCategory	@ValueMapValue String filterCategory

Demo Talking Point

"Custom Course Cards component with a three-field authoring dialog: heading, limit, and category filter. The HTL template invokes the Sling Model via data-sly-use and iterates the resulting CourseBean list. CSS clientlib delivers responsive card styling. Dialog values are auto-injected into the Sling Model via @ValueMapValue annotations."

8. Experience Fragments (XFs)

Concept

An Experience Fragment is a reusable, composable piece of authored content (like a header, footer, or hero banner) that can be embedded across multiple pages. XFs are stored under `/content/experience-fragments/` and are authored just like pages but never published as standalone pages.

Why Use XFs?

- **Single source of truth** — change the header in one place, updates everywhere
- **Reusability** — same XF can be embedded in multiple pages or even external channels
- **Separation of concerns** — different teams can own different XFs
- **Versioning** — XFs are versionable like pages
- **Variations** — one XF can have multiple variations (web, mobile, AEM Screens)

What Was Built

XF	Path	Contains
Header	/content/experience-fragments/genc-aem-training/site/Header	Site Header component (logo + nav)
Footer	/content/experience-fragments/genc-aem-training/site/Footer	Site Footer component (links + copyright)

XF Authoring Workflow

- Created folder structure: `/content/experience-fragments/genc-aem-training/site/`
- Created Header XF using the **Web Variation** template
- Dragged the custom **Site Header** component into the XF
- Configured the dialog: logo path + 4 navigation links via multifield
- Same process for Footer XF: dragged Site Footer component, configured 4 footer links and copyright

Header XF Configured Data

Field	Value
Logo	Sample image from DAM (we-retail)
Nav link 1	Home → /content/genc-aem-training/us/en/project/home-
Nav link 2	About → /content/genc-aem-training/us/en/project/about-us
Nav link 3	Courses → /content/genc-aem-training/us/en/project/courses-
Nav link 4	Contact → /content/genc-aem-training/us/en/project/contact-us

Footer XF Configured Data

Field	Value
Footer Link 1	Home → /content/genc-aem-training/us/en/project/home-
Footer Link 2	About Us → /content/genc-aem-training/us/en/project/about-us
Footer Link 3	Courses → /content/genc-aem-training/us/en/project/courses-
Footer Link 4	Contact Us → /content/genc-aem-training/us/en/project/contact-us
Copyright	© 2026 GenC Institute. All rights reserved.

Demo Talking Point

> *"Header and footer are managed as Experience Fragments — defined once, embedded in the page template, appears on all pages. Changing the header in the XF editor updates it across all four site pages automatically. This is a common pattern for site-wide consistency and demonstrates the separation between page-specific content and shared composition."*

9. Custom Header & Footer Components

Concept

While AEM provides out-of-the-box header/footer components, the case study required **custom components** with dialogs specifying: logo upload, navigation multifield, footer links multifield, and copyright text. Both components live in the **GenC AEM Training - Structure** component group.

Multifield: The Key Concept

A **multifield** is a Granite UI form widget that lets authors add a variable number of repeating items (e.g., "add as many navigation links as you want"). When `composite="true"`, each multifield entry is stored as a **child node** (item0, item1, item2, ...) rather than a single property.

■ **Note: HTL Reading Pattern:** Because composite multifield items are child nodes (not properties), HTL must use `data-sly-use.X="childNodeName"` to load the resource, then iterate its children using `data-sly-list.item="${X.children}"`.

9.1 — Header Component

File Structure

```
header/
■■■ .content.xml              # cq:Component, group: Structure
■■■ header.html               # HTL template
■■■ _cq_dialog/
■   ■■■ .content.xml          # Dialog with Logo tab and Navigation tab
■■■ _cq_template.xml          # Initial node template (optional but recommended)
```

.content.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<jcr:root xmlns:cq="http://www.day.com/jcr/cq/1.0"
          xmlns:jcr="http://www.jcp.org/jcr/1.0"
    jcr:primaryType="cq:Component"
    jcr:title="Site Header"
    jcr:description="Site-wide header with logo and navigation links."
    componentGroup="GenC AEM Training - Structure"/>
```

header.html (HTL Template)

```
<header class="cmp-header" data-sly-use.navItems="navLinks">

    <!-- Logo (image, when set) -->
    <div class="cmp-header__logo" data-sly-test="${properties.logoPath}">
        <a href="${request.contextPath}/content/.../home-.html"
           class="cmp-header__logo-link">
            <img src="${properties.logoPath}"
                 alt="${properties.logoAlt || 'Logo'}"
                 class="cmp-header__logo-img"/>
        </a>
    </div>

    <!-- Fallback text logo -->
    <div class="cmp-header__logo cmp-header__logo--text"
         data-sly-test="${!properties.logoPath}">
        <a href="${request.contextPath}/content/.../home-.html"
```

```
            class="cmp-header__logo-link">
             GenC Institute
        </a>
    </div>

    <!-- Navigation (multifield) -->
    <nav class="cmp-header__nav"
         data-sly-list.link="${navItems.children}">
        <a class="cmp-header__nav-link"
           href="${link.linkURL}.html"
           target="${link.openInNewTab ? '_blank' : '_self'}"
           rel="${link.openInNewTab ? 'noopener noreferrer' : ''}">
            ${link.linkText}
        </a>
    </nav>
</header>
```

Dialog Fields (_cq_dialog/.content.xml)

Tab	Field	Type	Stores As
Logo	Logo Image	pathfield (rooted at /content/dam)	./logoPath
Logo	Logo Alt Text	textfield	./logoAlt
Navigation	Navigation Links	multifield (composite=true)	./navLinks/item0, item1, ...
Nav item	Link Text	textfield (required)	./linkText
Nav item	Link URL	pathfield	./linkURL
Nav item	Open in new tab	checkbox	./openInNewTab

9.2 — Footer Component

File Structure

```
footer/
■■■ .content.xml              # cq:Component, group: Structure
■■■ footer.html               # HTL template
■■■ _cq_dialog/
■   ■■■ .content.xml          # Dialog with Links tab and Copyright tab
■■■ _cq_template.xml          # CRITICAL — pre-creates footerLinks node
```

footer.html (HTL Template)

```
<footer class="cmp-footer" data-sly-use.footerLinkItems="footerLinks">

    <ul class="cmp-footer__links"
        data-sly-list.link="${footerLinkItems.children}">
        <li class="cmp-footer__links-item">
            <a class="cmp-footer__links-link"
               href="${link.linkURL}.html">
                ${link.linkText}
            </a>
        </li>
    </ul>

    <p class="cmp-footer__copyright"
```

```
        data-sly-test="${properties.copyrightText}">
         ${properties.copyrightText}
    </p>

</footer>
```

_cq_template.xml (Critical for Footer)

Without this file, dropping a fresh Footer component throws `SightlyException: No use provider could resolve identifier footerLinks` because the child node doesn't exist yet. The `_cq_template.xml` defines the initial state of the component when first added to a page — pre-creating the empty `footerLinks` child node so HTL can load it successfully.

```
<?xml version="1.0" encoding="UTF-8"?>
<jcr:root xmlns:cq="http://www.day.com/jcr/cq/1.0"
          xmlns:jcr="http://www.jcp.org/jcr/1.0"
          xmlns:nt="http://www.jcp.org/jcr/nt/1.0"
          xmlns:sling="http://sling.apache.org/jcr/sling/1.0"
    jcr:primaryType="nt:unstructured"
    sling:resourceType="genc-aem-training/components/footer">
    <footerLinks jcr:primaryType="nt:unstructured"/>
</jcr:root>
```

Dialog Fields (_cq_dialog/.content.xml)

Tab	Field	Type	Stores As
Footer Links	Footer Links	multifield (composite=true)	./footerLinks/item0, ...
Link item	Link Text	textfield (required)	./linkText
Link item	Link URL	pathfield	./linkURL
Copyright	Copyright Text	textfield	./copyrightText

Demo Talking Point

> *"Custom Header and Footer components with multifield dialogs let authors manage navigation links and footer links visually. The composite multifield stores each entry as a child node, which HTL iterates using data-sly-use plus children. The _cq_template.xml is a defensive pattern: it initializes the multifield resource on first component drop, preventing render errors before authoring takes place."*

10. Editable Template (Content Page)

Concept

An **Editable Template** defines the structure of pages — what's fixed (header, footer, locked components) and what's flexible (where authors can drop content). Templates live under `/conf/<config>/settings/wcm/templates/`.

Template Used

Property	Value
Template name	Content Page
Path	/conf/genc-aem-training/settings/wcm/templates/content-page
Status	Enabled
Used by	All 4 site pages (Home, About Us, Courses, Contact Us)

Template Structure

From top to bottom of any page using this template:

Position	Element	Configured To
Top	Experience Fragment placeholder	Header XF (/content/.../site/Header/master)
Middle	Layout Container (responsive grid)	Open for authors to drop content
Bottom	Experience Fragment placeholder	Footer XF (/content/.../site/Footer/master)

Template Policies

Policies define which components authors can place in each container. The main Layout Container policy includes:

- Course Cards (our custom component)
- Core Components (Title, Text, Teaser, Image, Tabs, Container, etc.)
- Experience Fragment
- Content Fragment List
- Carousel
- Button
- Embed

Why Editable Templates Matter

- **Governance**: Authors can't drop just any component anywhere
- **Consistency**: Header/footer locked at template level — appears on every page

- **Flexibility**: Content area remains author-editable
- **Maintainability**: Template changes propagate to all pages using it

Demo Talking Point

> *"Editable template controls page structure. The Header and Footer Experience Fragment placeholders are locked at the template level, ensuring all four pages have consistent branding. Template policies restrict which components authors can place — providing governance with flexibility. This pattern keeps authors productive while preventing site-wide inconsistencies."*

11. Site Pages

Site Structure

```
/content/genc-aem-training/
■■■ us/
    ■■■ en/
        ■■■ project/                    # Site root
            ■■■ home-                    # Home page
            ■■■ about-us                 # About Us page
            ■■■ courses-                 # Courses listing
            ■■■ contact-us               # Contact Us
```

Pages Created

#	Page	URL	Key Content
1	Home	/content/genc-aem-training/us/en/project/home-	Hero, Image+Text, Course Cards (featured)
2	About Us	/content/genc-aem-training/us/en/project/about-us	Institute info, history
3	Courses	/content/genc-aem-training/us/en/project/courses-	Title + Course Cards (all 12, no limit)
4	Contact Us	/content/genc-aem-training/us/en/project/contact-us	Address, phone, email

Common Features (via Template)

- All pages inherit the Header XF (top) and Footer XF (bottom)
- All pages use the Content Page template
- Page-specific content authored in the main Layout Container
- Same blue/white brand styling via clientlibs

Demo Talking Point

> *"Four pages all using the same Content Page template. The header and footer come from the template's XF placeholders, so they're consistent across all pages. Page-specific content lives in the main layout container — each page has its own purpose but shares the site's structure and branding."*

12. Clientlibs (CSS/JS)

Concept

Client Libraries are AEM's mechanism for delivering CSS and JavaScript to pages. Clientlibs are categorized and loaded based on the categories list, allowing modular CSS/JS bundles that can be selectively included by templates or pages.

What Was Built

Property	Value
Path	ui.apps/.../clientlibs/clientlib-site/
Category	genc-aem-training.site
CSS files	coursecards.css, header-footer.css
JS files	(none)

File Structure

```
clientlib-site/
■■■ .content.xml              # categories=[genc-aem-training.site]
■■■ css.txt                   # manifest listing CSS files
■■■ js.txt                    # manifest listing JS files (empty)
■■■ css/
    ■■■ coursecards.css       # styling for Course Cards
    ■■■ header-footer.css     # styling for Header and Footer
```

.content.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<jcr:root xmlns:cq="http://www.day.com/jcr/cq/1.0"
          xmlns:jcr="http://www.jcp.org/jcr/1.0"
    jcr:primaryType="cq:ClientLibraryFolder"
    allowProxy="{Boolean}true"
    categories="[genc-aem-training.site]"/>
```

css.txt

```
# (Apache license header)
#base=css
coursecards.css
header-footer.css
```

How Pages Load the Clientlib

The page template (via its `customheaderlibs.html` or `page.html`) loads the clientlib by referencing the category `genc-aem-training.site`. All CSS files listed in `css.txt` are concatenated and served as a single file in production, minified when possible.

CSS Highlights — Course Cards

- Responsive grid: `repeat(auto-fill, minmax(300px, 1fr))`
- Blue gradient category badges with proper letter-spacing

- Subtle hover lift animation
- Mobile-first breakpoint at 768px
- Navy blue (#003366) brand color throughout

CSS Highlights — Header & Footer

- Header: blue gradient (135deg, #003366 → #0066cc)
- Flexbox for logo-left, nav-right alignment
- Footer: dark theme (#1a1a1a) with centered link list
- Hover effects on nav links
- Responsive collapse on mobile

Demo Talking Point

> *"Clientlibs allow modular CSS and JavaScript delivery. The category attribute lets pages opt-in to specific style bundles, enabling efficient resource loading. The css.txt manifest defines load order. In production, AEM minifies and concatenates files for performance."*

13. Users, Groups & ACLs (RBAC)

Concept

Role-Based Access Control (RBAC) restricts what different users can do. AEM uses Groups to define roles, Users belong to Groups, and Access Control Lists (ACLs) grant or deny privileges on JCR paths. Permission inheritance flows downward.

Groups Created

Group Name	Role	Member of (parent)
gencinstitute-authors	Regular content authors	authors
gencinstitute-xf-admins	Experience Fragment administrators	authors
gencinstitute-readers	Read-only users	contributors

Users Created

User ID	Group Assignment	Notes
author1	gencinstitute-authors	Demo author user
admin-xf1	gencinstitute-xf-admins	Can manage Experience Fragments
reader1	gencinstitute-readers	Read-only demo user

Permission Matrix

Path	Authors	XF Admins	Readers
/content/genc-aem-training (pages)	Read/Write/Delete/Replicate	Read/Write/Delete/Replicate	Read-only
/content/dam/genc-aem-training (DAM)	Read/Write/Delete/Replicate	Read/Write/Delete/Replicate	Read-only
/content/experience-fragments/genc-aem-training (XFs)	Read-only (DENIED)	Read/Write/Delete/Replicate	Read-only

JCR Privilege Mapping

UI Label	JCR Privilege	What It Grants
Read	jcr:read	View nodes and properties
Modify	jcr:modifyProperties	Edit property values
Create	jcr:addChildNodes	Add child nodes
Delete	jcr:removeNode / jcr:removeChildNodes	Delete nodes

UI Label	JCR Privilege	What It Grants
Read ACL	jcr:readAccessControl	View permissions
Edit ACL	jcr:modifyAccessControl	Change permissions
Replicate	crx:replicate	Publish to publisher (AEM-specific, not in jcr:all)

Important: jcr:all vs Replicate

■ **Note: jcr:all** aggregates all JCR-level privileges (read, write, delete, ACL management, etc.) but does **NOT** include **crx:replicate**. Replication is an AEM-specific privilege outside the JCR core spec. For users who need to publish, both must be granted.

Setup Approach Used

UI-based via the AEM User Admin console and Permissions tool. This was chosen over Repoinit (code-based) for speed and demo simplicity. The Repoinit equivalent could be added later for source-controlled, repeatable deployment.

Testing Each User

- **Test 1 (author1):** Login as author1 → try to edit Header XF → should be denied
- **Test 2 (admin-xf1):** Login as admin-xf1 → edit Header XF → should succeed
- **Test 3 (reader1):** Login as reader1 → browse pages → Edit button should be hidden

Demo Talking Point

> *"I implemented role-based access control with three distinct roles. Regular authors have full content rights on pages and DAM but are explicitly denied write access on Experience Fragments — preventing accidental modification of site-wide headers and footers. A separate XF admin role has full XF management privileges. Readers have read-only access. This demonstrates separation of duties and least-privilege principles. The permission model uses granular JCR privileges plus the AEM-specific crx:replicate for publish rights."*

14. Maven Build & Deployment

Project Modules

Module	Purpose
genc-aem-training (root)	Parent POM aggregating all submodules
core	Java code: Sling Models, OSGi services, beans
ui.apps	Components, clientlibs, templates (deploys to /apps)
ui.apps.structure	Defines allowed root paths for ui.apps validation
ui.config	OSGi configurations as .cfg.json files
ui.content	Content packages: CF Model, pages, tags, XFs
all	Aggregates everything into one deployable package
it.tests	Integration tests (passes)
ui.tests	Cypress UI tests (skipped — SSL cert issue)

Build Command

```
mvn clean install -PautoInstallSinglePackage
```

What Each Phase Does

Phase	Action
clean	Removes target/ from every module
compile	Compiles all Java code in core
package	Builds JAR/ZIP files for each module
install (local)	Installs to local Maven repository
install (AEM, via -P profile)	Uploads and installs the package to localhost:4502

filter.xml — Critical Files

Each module's `filter.xml` defines which JCR paths the module owns and deploys. Getting these right is critical for clean deployment.

ui.apps/META-INF/vault/filter.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<workspaceFilter version="1.0">
    <filter root="/apps/genc-aem-training/clientlibs"/>
    <filter root="/apps/genc-aem-training/components"/>
    <filter root="/apps/genc-aem-training/i18n"/>
```

```
</workspaceFilter>
```

ui.content/META-INF/vault/filter.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<workspaceFilter version="1.0">
    <filter root="/conf/genc-aem-training" mode="merge"/>
    <filter root="/content/genc-aem-training" mode="merge"/>
    <filter root="/content/dam/genc-aem-training" mode="merge"/>
    <filter root="/content/experience-fragments/genc-aem-training" mode="merge"/>
    <filter root="/content/cq:tags/genc-aem_training" mode="merge"/>
</workspaceFilter>
```

mode="merge"

The `mode="merge"` attribute means: when the package is installed, existing content at these paths is preserved (merged), not deleted. This is essential for content paths so authored data isn't wiped on each deploy.

Demo Talking Point

> *"The Maven build follows standard AEM conventions with separation between immutable code (ui.apps deploys to /apps) and mutable content (ui.content deploys to /content and /conf). The autoInstallSinglePackage profile uploads the complete package to AEM via its package manager API. Filter files with mode='merge' protect authored content from being overwritten on redeploys."*

15. Architectural Patterns Used

This project demonstrates several established AEM and software engineering patterns:

Pattern	Where Used	Why
Configuration as Code	OSGi .cfg.json files	Reproducible deployment, in source control
Service Locator (OSGi)	CourseConfigService	Loose coupling, testable
Sling Model	CourseListModel	Decouples content (JCR) from presentation (HTL)
POJO Data Container	CourseBean	Clean data carrier, easy to test
Pre-formatting in Java	Date strings in CourseBean	Avoids HTL Calendar issues
Strategy Pattern (fallback)	Tag resolution with capitalize fallback	Robustness against missing tags
Multifield composite	Header/Footer dialogs	Allow variable number of repeating items
Initialization template	_cq_template.xml in Footer	Prevent HTL crashes on empty drops
Experience Fragment	Header XF, Footer XF	Single source of truth for shared content
Editable Template	Content Page	Locked structure + flexible content
Clientlib	clientlib-site	Modular CSS delivery, performance
RBAC	Three custom groups with explicit ACLs	Separation of duties, least privilege

Senior-Developer Practices Demonstrated

- **Configurability over hard-coding** — DAM path is OSGi-configurable
- **Defensive coding** — Null checks in Sling Model, fallback tag resolution
- **Logging** — SLF4J logger in CourseListModel for debugging
- **Separation of concerns** — Each module/class has a single responsibility
- **Documentation** — Javadoc on all public classes and methods
- **Security awareness** — Explicit DENY on XF path for regular authors

16. Issues Encountered & Fixes

A real engineering journey isn't bug-free. Documenting the issues hit during development is valuable both for learning and to show problem-solving skills during demos.

Issue	Cause	Fix
Malformed filter.xml with <__workspaceFilter__>	Typo from manual edit	Replaced with <workspaceFilter>
Filter validation error: /conf not allowed in ui.apps	Wrong module ownership	Moved /conf and /cq:tags filters to ui.content
Missing HelloWorldModel.java	Archetype scaffolding left behind	Deleted the helloworld component folder
Maven can't delete generated HTL Java file	Eclipse holding file lock	Disabled Eclipse auto-build, manually deleted target/
HTL '>' not allowed inside ${} expression	HTL parser limitation	Simplified template, removed comparison
Course dates rendering as Calendar toString	HTL @format on Calendar doesn't work cleanly	Pre-formatted as String in Sling Model
Course categoryTitle empty (TagManager returns null)	Tags namespace path mismatch	Added fallback to extract from tag ID string
Multifield items not rendering in HTL	Composite multifield stores as child nodes, not properties	Used data-sly-use + children iteration
Footer crash: 'No use provider could resolve footerLinks'	Empty child node on fresh drop	Added _cq_template.xml to pre-create node
HTL: 'mismatched input ('	Method calls like resource.getChild() not allowed in HTL 1.4	Used data-sly-use pattern instead
Cypress UI tests fail on every build	SSL cert verification can't reach nodejs.org	Ignored — not relevant to demo

Lessons Learned

- **HTL parser is strict** — no XML entities (>) and no method calls with args inside ${}
- **Multifield child-node pattern** is a notorious gotcha — always use data-sly-use
- **_cq_template.xml** is essential for components using data-sly-use on child resources
- **Eclipse auto-build** can lock files — disable it during Maven runs
- **Filter modes matter** — always use mode="merge" on content paths
- **Tags can disappear** — store them properly in ui.content for source-control

17. Quick Reference URLs

Useful URLs for navigating and verifying the AEM environment:

Purpose	URL
AEM Author start	http://localhost:4502
Sites console	/sites.html/content/genc-aem-training
Assets / DAM	/assets.html/content/dam/genc-aem-training/courses
Content Fragment Models	/mnt/overlay/dam/cfm/models/console/content/models.html/conf/genc-aem-training
Tags console	/aem/tags.html/content/cq:tags
Templates console	/libs/wcm/core/content/sites/templates.html/conf/genc-aem-training
Experience Fragments	/aem/experience-fragments.html/content/experience-fragments/genc-aem-training/site
OSGi Configuration Manager	/system/console/configMgr
OSGi Components	/system/console/components
OSGi Bundles	/system/console/bundles
Sling Models registry	/system/console/status-slingmodels
CRXDE Lite	/crx/de/index.jsp
User Admin	/libs/granite/security/content/v2/userEditor.html
Group Admin	/libs/granite/security/content/v2/groupEditor.html
Permissions Editor	/libs/granite/security/content/v2/permissionEditor.html
Logout	/system/sling/logout

Page URLs (for direct preview)

Page	URL
Home	/editor.html/content/genc-aem-training/us/en/project/home-.html
About Us	/editor.html/content/genc-aem-training/us/en/project/about-us.html
Courses	/editor.html/content/genc-aem-training/us/en/project/courses-.html
Contact Us	/editor.html/content/genc-aem-training/us/en/project/contact-us.html
Header XF (edit)	/editor.html/content/experience-fragments/genc-aem-training/site/Header/master.html
Footer XF (edit)	/editor.html/content/experience-fragments/genc-aem-training/site/Footer/master.html

18. Demo Flow Suggestions

Recommended Demo Order (15–20 mins)

1. Open Tags Console

Show /content/cq:tags/genc-aem_training with the 10 category tags. **Talking point:** "This is our content taxonomy — categories that authors can apply."

2. Open Content Fragment Model

Show the Course model with its 6 fields. Highlight that the Tag field is scoped. **Talking point:** "The schema for our courses — strongly typed, reusable."

3. Open one Content Fragment

Show B.Tech Computer Science. Highlight fields populated. **Talking point:** "An instance of the model with real data. Authors edit this; developers don't."

4. Open OSGi configMgr

Find 'GenC AEM Training - Course Configuration'. Show DAM path and limit values. **Talking point:** "Configurable at runtime via OSGi — change the DAM path without redeploying."

5. Open Sling Models status

/system/console/bundles → confirm bundle Active. **Talking point:** "My Sling Model is registered and active. It reads fragments and feeds the component."

6. Open the Courses page

Show all 12 cards rendering. Click one — show the dialog with limit and filter. **Talking point:** "All 12 courses displayed via my Course Cards component. Authors can limit and filter via the dialog."

7. Demo the filter

Change filter to 'Engineering' — show only Engineering courses appear. **Talking point:** "Authors can dynamically filter without touching code."

8. Open the Header XF

Show the configured header with logo and nav links. **Talking point:** "Site-wide header managed as an Experience Fragment."

9. Open the Footer XF

Show the configured footer. **Talking point:** "Same pattern for footer — change once, affects all pages."

10. Open the template editor

Show the Content Page template structure with XF placeholders. **Talking point:** "Editable template with locked header/footer XFs and flexible content area."

11. Show navigation to another page

Click About Us in the header — both header and footer remain consistent. **Talking point:** "Common header/footer across all pages, as required."

12. Demo RBAC

Logout admin, login as author1. Try to edit Header XF — denied. Logout, login as admin-xf1 — can edit. **Talking point:** "Role-based access control: regular authors can't break site-wide content; XF admins can."

Anticipated Q&A;

Question	Answer
Why an OSGi service instead of hardcoding the path?	Runtime configurability + testability. Admins can change paths per environment via We
Why a Sling Model instead of a JSP scriptlet?	Modern best practice. Sling Models provide dependency injection, are testable, and cl
Why pre-format dates in Java?	HTL has limitations with Calendar objects — truthy-checks fail unexpectedly. Pre-form
Why use Experience Fragments for header/footer?	Single source of truth. Change once, applies everywhere. Also supports variations (we
Why a custom component instead of OOTB Header/Footer?	The OOTB Header/Footer doesn't support multifield for navigation links — that requires a custom dialo
What does the _cq_template.xml file do?	It defines the initial node structure when the component is first dropped on a page. We
Why separate authors and XF admins?	Separation of duties. Most authors handle daily content; XF admins handle site-wide e

Closing Statement

> *"This case study demonstrates the full AEM development stack: content modeling with Tags and Content Fragments; configurable runtime behavior via OSGi services; business logic in Sling Models; presentation via custom HTL components with Granite UI dialogs; site-wide composition via Experience Fragments and editable templates; and security via role-based access control. The architecture follows AEM best practices and demonstrates senior-developer patterns: configurability over hard-coding, separation of concerns, defensive coding, and least-privilege security."*

End of Documentation

Total artifacts: 12 fragments, 1 OSGi service, 1 Sling Model, 3 custom components, 2 Experience Fragments, 4 pages, 3 user groups, multiple ACLs.